

```

    release: LPU_device_release,
}

```

You may note that some modern drivers use another format, as shown below:

```

struct file_operations LPU_ops = {
    open = LPU_device_open,
    read = LPU_device_read,
    write = LPU_device_write,
    release = LPU_device_release
};

```

This is supported by a GCC extension and is helpful if you want to port your device driver. You can use the old format for this tutorial.

Similarly, `/dev/` is represented by a file structure (you may look at `linux/fs.h` for more details). There are a few points that you should note here. The 'file' (in this context) is a kernel-level structure and never goes to the user-space program level. But you should not mistake this for FILE, which is defined by the glibc library and is not a part of the kernel-space function. Our 'file' is not a file on the disk but rather an 'abstract open file', represented by `inode (struct)`.

Also, we have the `inode_operations`. For your information, here is the `struct inode_operations`:

```

struct inode_operations {
    int (*create) (struct inode *, struct dentry *, int, struct nameidata *);
    struct dentry * (*lookup) (struct inode *, struct dentry *, struct
nameidata *);

    int (*link) (struct dentry *, struct inode *, struct dentry *);
    int (*unlink) (struct inode *, struct dentry *);
    int (*symlink) (struct inode *, struct dentry *, const char *);
    int (*mknod) (struct inode *, struct dentry *, int);
    int (*mkdir) (struct inode *, struct dentry *, int);
    int (*rmdir) (struct inode *, struct dentry *);
    int (*mknod) (struct inode *, struct dentry *, int, dev_t);
    int (*rename) (struct inode *, struct dentry *,
        struct inode *, struct dentry *);

    int (*readlink) (struct dentry *, char __user *, int);
    void * (*follow_link) (struct dentry *, struct nameidata *);
    void (*put_link) (struct dentry *, struct nameidata *, void *);
    void (*truncate) (struct inode *);
    int (*permssion) (struct inode *, int);
    int (*setattr) (struct dentry *, struct iattr *);
    int (*getattr) (struct vfsmount *, struct dentry *, struct kstat *);
    int (*setattr) (struct dentry *, const char *, const void *, size_t, int);
    ssize_t (*getxattr) (struct dentry *, const char *, void *, size_t);
    ssize_t (*listxattr) (struct dentry *, char *, size_t);
    int (*removexattr) (struct dentry *, const char *);
    void (*truncate_range) (struct inode *, loff_t, loff_t);
    long (*fallocate) (struct inode *, int mode, loff_t offset,
        loff_t len);
    int (*femap) (struct inode *, struct fmem_extent_info *, u64 start,
        u64 len);
}

```

Another important one is `filep`, which is the instance of `file (struct)`:

```

extern int __f_setown(struct file *fip, struct pid *, enum pid_type, int force);

```

Here is the relevant code corresponding to the `struct file`:

```

struct file {
    /*
     * fu_list becomes invalid after file_free is called and queued via
     * fu_rcuhead for RCU freeing
     */
    union {
        struct list_head fu_list;
        struct rcu_head fu_rcuhead;
    } fu;
    struct path f_path;
#define f_dentry f_path.dentry
#define f_vfsnt f_path.mnt
    const struct file_operations *f_op;
    spinlock_t f_lock; /* f_op_links, f_flags, no IRQ */
    atomic_long_t f_count;
    unsigned int f_flags;
    fmode_t f_mode;
    loff_t f_pos;
    struct fown_struct f_owner;
    const struct cred *f_cred;
    struct file_ra_state f_ra;

    u64 f_version;

#ifdef CONFIG_SECURITY
    void *f_security;
#endif

    /* needed for tty driver, and maybe others */
    void *private_data;

#ifdef CONFIG_EPOLL
    /* Used by fs/eventpoll.c to link all the hooks to this file */
    struct list_head f_ep_links;
#endif

#ifdef CONFIG_SECURITY
    struct address_space *f_mapping;
#endif
#ifdef CONFIG_DEBUG_WRITECOUNT
    unsigned long f_mnt_write_state;
#endif
}

```

You may have noticed that we have used a major number in our code. This is used while registering the character device:

```

extern int register_chrdev_region(dev_t, unsigned, const char *);
extern int register_chrdev(unsigned int, const char *,
    const struct file_operations *);

```

The major number is employed to indicate the driver